Final Project Report

Project 3: Enumeration Deep-Dive

Student Name	**Steve**
Project	Project 3 — Enumeration Deep-Dive
Specialization	Network Scanning and Enumeration
Scanner Machine	Kali Linux (hostname: V2310) — 192.168.10.74
Target Machines	192.168.10.71 — Windows Server VM (domain controller, steve.local); 192.168.10.33 — Windows VM
Tools Used	Nmap Scripting Engine (NSE), enum4linux, dig, snmpwalk

1. Objective

Enumeration means asking a machine polite questions and recording how much it answers without any password. In this project three services on the lab targets are enumerated — SMB (shared folders and user accounts), DNS (zone transfer) and SNMP (device information) — and for every piece of information that leaks, a hardening fix is written to stop the leak. All work was done on the student's own lab virtual machines.

2. Tools Used

Tool	Purpose
Nmap Scripting Engine (NSE)	Ready-made scripts that query services, e.g. smb-enum-shares, smb-enum-users, snmp-info
enum4linux	One command that pulls everything SMB will give: shares, users, groups, password policy
dig	Asks DNS questions; with axfr it requests a full zone transfer (the entire list of names in the domain)
snmpwalk	Walks through an SNMP agent's information tree using a community string (the default is 'public')

3. Procedure

Setup: Kali and both Windows VMs switched on and on the same network; the targets were checked reachable before scanning. Every scan was run from Kali; the Windows machines were only targets, except Step 5, where SNMP was deliberately switched on inside the student's own .33 VM so the SNMP enumeration could be practised.

Step 1 — SMB enumeration with Nmap scripts (target .71):

```
nmap --script smb-enum-shares,smb-enum-users -p 445 192.168.10.71 -oN enum-smb-nmap.txt
```

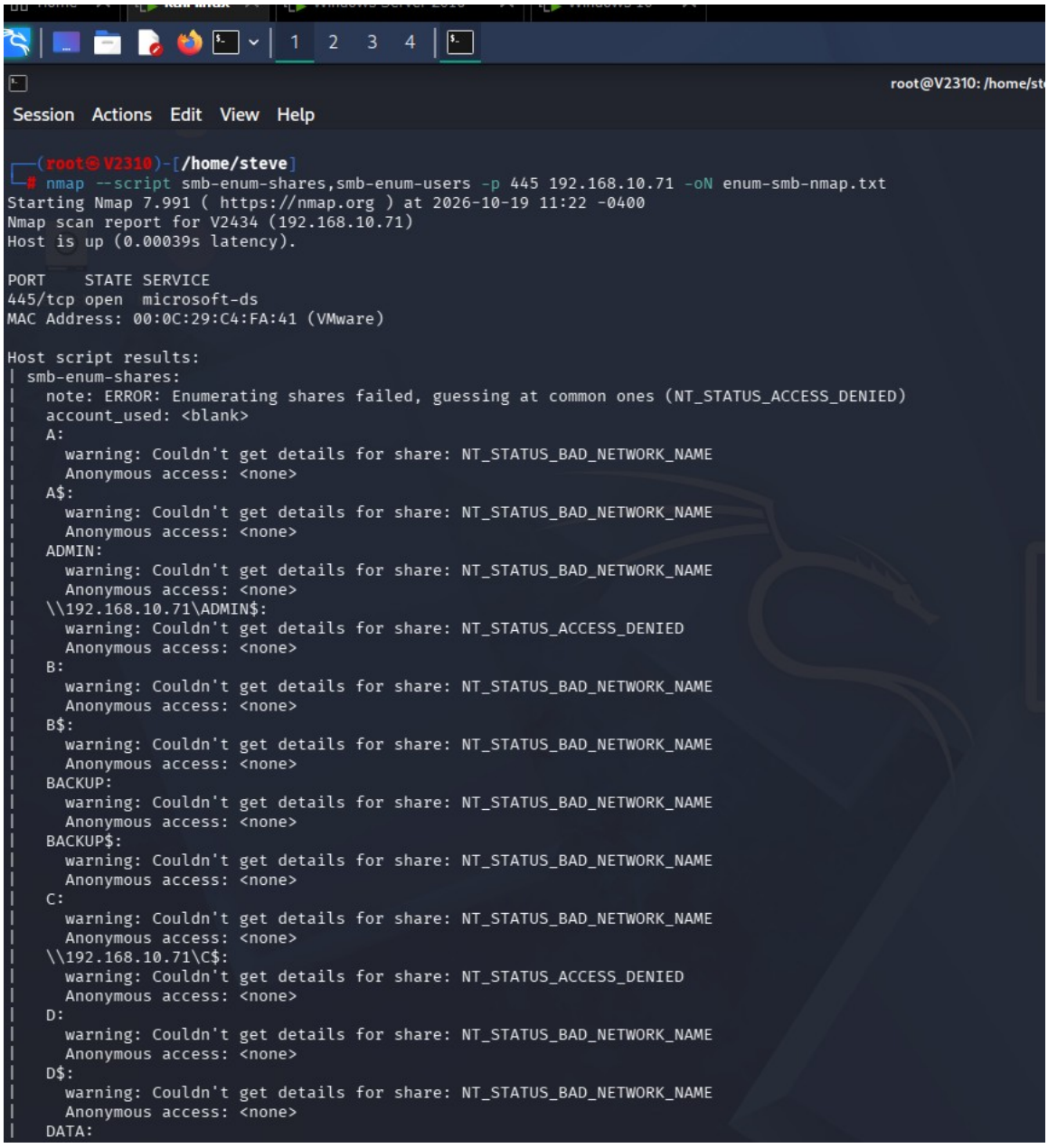


Figure 1 — Step 1: Nmap SMB enumeration scripts against .71.

What this screenshot shows: Port 445 is open, but the share list could not be enumerated anonymously: the script reports 'Enumerating shares failed (NT_STATUS_ACCESS_DENIED)' with

a blank account, and fell back to guessing common share names. Two error messages appear, and the difference matters: NT_STATUS_BAD_NETWORK_NAME means the guessed share does not exist (A, B, BACKUP and similar guesses), while NT_STATUS_ACCESS_DENIED on ADMIN$ and C$ means those shares exist but refuse anonymous access. Every entry shows 'Anonymous access: <none>' — no share contents, users or details leaked.

Step 2 — SMB enumeration with enum4linux (target .71):

```
enum4linux -a 192.168.10.71 | tee enum-smb-full.txt
```

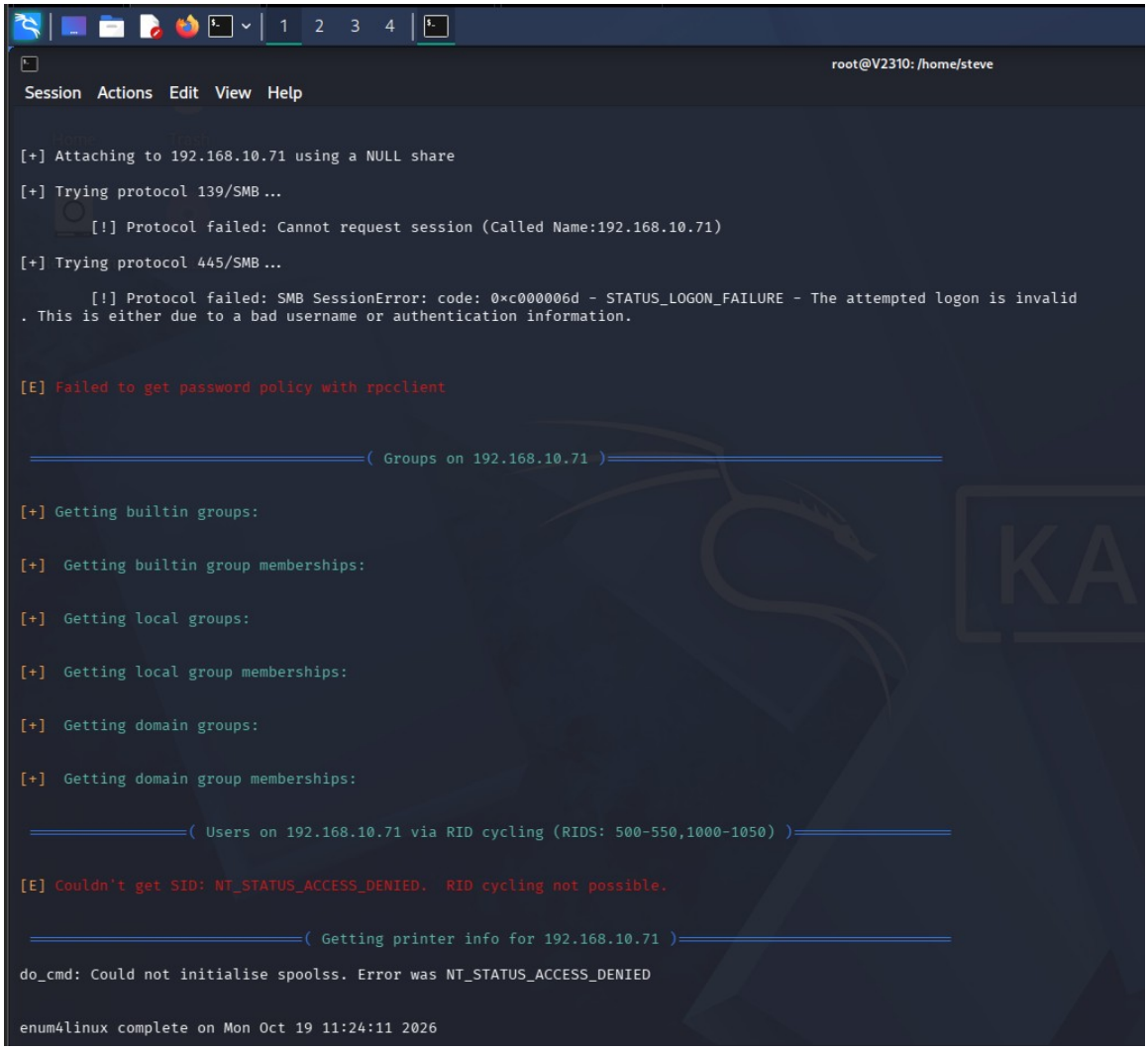


Figure 2 — Step 2: enum4linux against .71 — NULL session refused, all queries denied.

What this screenshot shows: The enum4linux run against .71. At the top it tries to attach with a NULL (no-password) session: the session request on port 139 fails, and the SMB logon on port 445 is rejected with STATUS_LOGON_FAILURE (code 0xc000006d) — the server does not accept the invalid logon. Everything after that is also refused: the password policy query failed, all group sections (built-in, local and domain) returned empty, user enumeration by RID cycling was impossible because the server would not give out its SID ('Couldn't get SID: NT_STATUS_ACCESS_DENIED'), and printer information was denied too. The tool completed

normally ('enum4linux complete') — the denials are the server's security working as designed, not a failed scan.

Step 3 — DNS zone transfer attempt (target .71, domain steve.local):

```
dig axfr @192.168.10.71 steve.local | tee dns-zone-transfer.txt
```



Figure 3 — Step 3: DNS zone transfer attempt against .71.

What this screenshot shows: dig confirms a DNS server exists at .71 ('1 server found'), but the full zone transfer is refused: 'Transfer failed.' The server only gives zone data to authorised name servers, so an unauthorised machine receives nothing. This is the correct, hardened result.

Step 4 — SNMP check on both targets:

```
sudo nmap -sU -p 161 --script snmp-info 192.168.10.71 -oN enum-snmp-check.txt
```

```
sudo nmap -sU -p 161 --script snmp-info 192.168.10.33 -oN enum-snmp-check-33.txt
```



Figure 4 — Step 4: SNMP check on .71 — port 161 closed.

What this screenshot shows: UDP port 161 (SNMP) is closed on the server .71: no SNMP service is running, so there is nothing to enumerate.

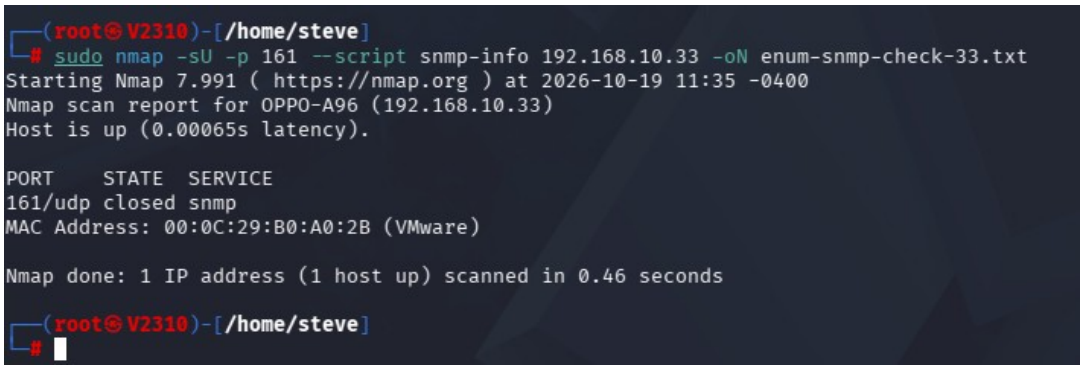


Figure 5 — Step 4: SNMP check on .33 — port 161 closed.

What this screenshot shows: The second target gives the same result: 161/udp closed on .33 as well. SNMP is switched off by default on both Windows machines, so to practise the SNMP part of this project the service was enabled on the student's own .33 VM in the next step.

Step 5 — Enabling SNMP on the student's own VM (.33) for the lab exercise:

On .33, the SNMP feature was installed with the administrator PowerShell command:

```
Add-WindowsCapability -Online -Name "SNMP.Client~~~~0.0.1.0"
```

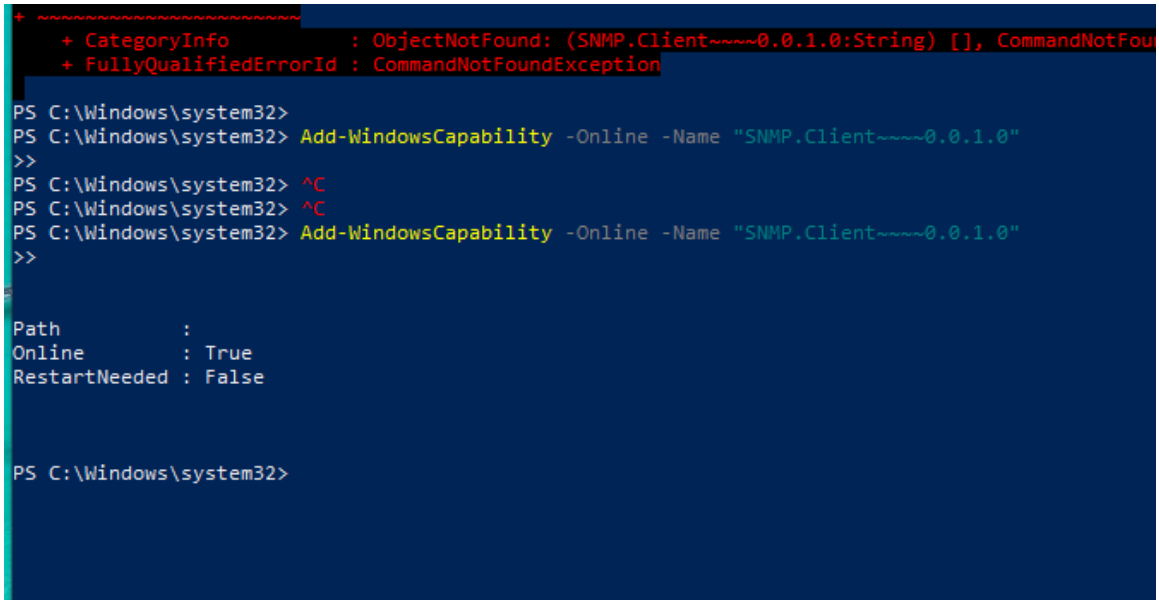


Figure 6 — Step 5: SNMP feature installed on .33 (Online: True, RestartNeeded: False).

What this screenshot shows: The capability install result on .33: 'Online: True' confirms the SNMP Service was installed successfully and no restart was needed. The service was then configured in services.msc with the community string 'public' (accepted community name, READ ONLY rights, accept packets from any host) and started — deliberately using the well-known default community, exactly the weak configuration this project tests for. The first walk attempt timed out because the community name had not been saved in the list; once 'public' was added properly and the service restarted, the walk in Step 7 succeeded.

Step 6 — SNMP re-check on .33 after enabling:

```
sudo nmap -sU -p 161 --script snmp-info 192.168.10.33 -oN enum-snmp-check-33.txt
```



Figure 7 — Step 6: SNMP re-check on .33 — port now open|filtered.

What this screenshot shows: The same check that reported 'closed' before now reports '161/udp open|filtered' — the change proves the SNMP service is now running on .33 (a closed port answers with an unreachable message; a listening or firewalled UDP service stays silent, which is why Nmap can only say open|filtered).

Step 7 — SNMP walk on .33 with the default community string:

```
snmpwalk -v2c -c public 192.168.10.33 | tee snmp-walk.txt
```

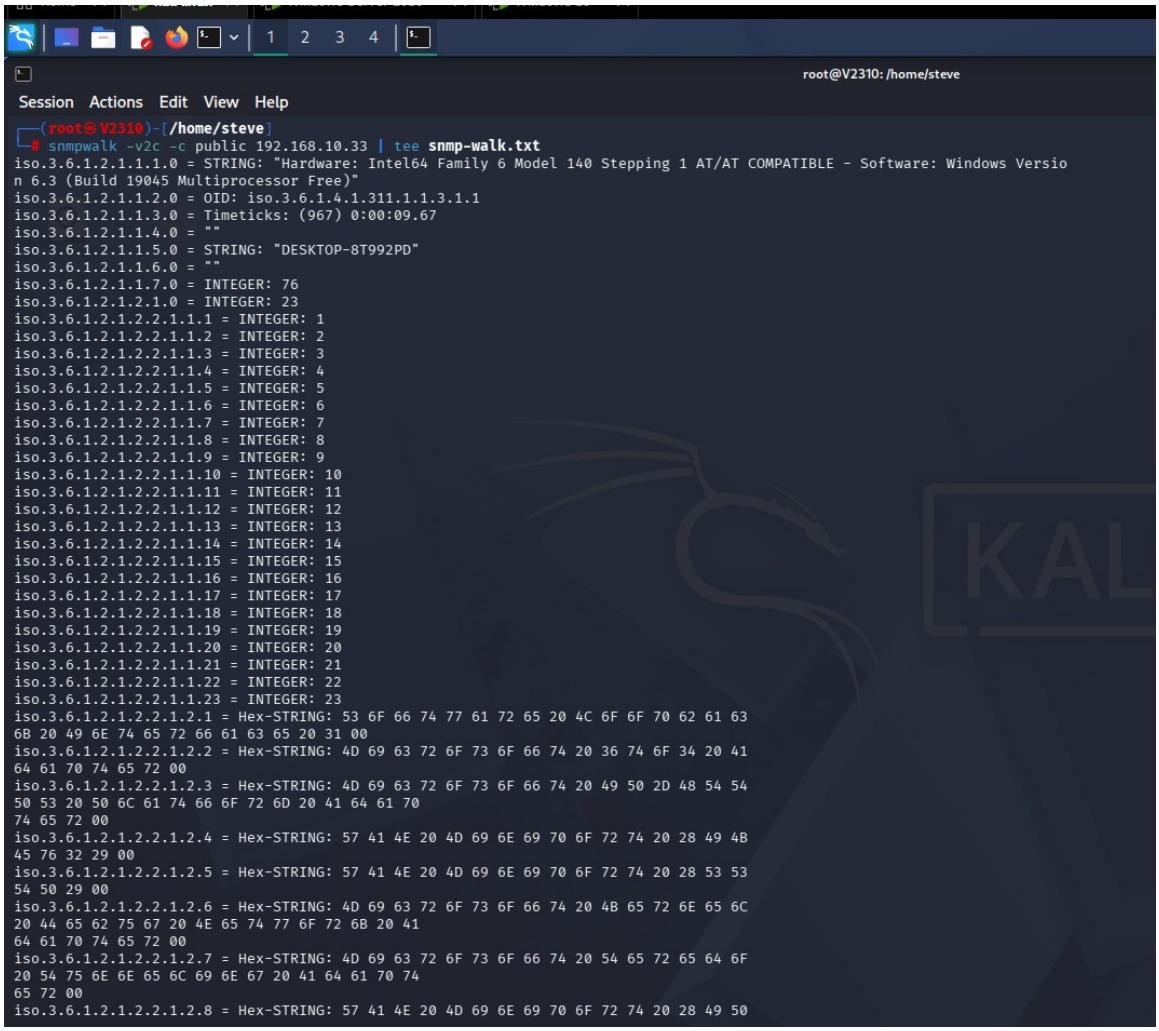


Figure 8 — Step 7: start of the snmpwalk output from .33 (full output saved in snmp-walk.txt).

What this screenshot shows: The walk succeeds with nothing but the word 'public'. The very first line (sysDescr) already leaks the machine's identity: 'Hardware: Intel64 Family 6 Model 140... Software: Windows Version 6.3 (Build 19045)' — a 64-bit Windows system, build 19045. Further lines give the system name (DESKTOP-8T992PD), the service uptime counter, and the network interface list (the Hex-STRING entries decode to names such as 'Software Loopback Interface 1' and 'Microsoft Wi-Fi Direct Virtual Adapter'), and the full saved output continues through running processes and installed software. All of this was available to any machine on the network that asked with the default community string.

4. Findings

#	Service	Target	What Was Found	Evidence	Risk
1	SMB	192.168.10.71	Anonymous share enumeration denied (NT_STATUS_ACCESS_DENIED). The script fell back to guessing share names; only the existence of the default admin shares C$ and ADMIN$ could be inferred, because they answer ACCESS_DENIED while non-existent names answer BAD_NETWORK_NAME. Anonymous access to every share: <none>.	Figure 1	Low
2	SMB	192.168.10.71	Password policy, group lists, printer info and the user list (RID cycling) all refused to an anonymous user; the server would not disclose its SID. No user or policy information leaked.	Figure 2	Low

3	DNS	192.168.10.71	Zone transfer (AXFR) for steve.local from an unauthorised machine refused — 'Transfer failed'. Full record list not disclosed.	Figure 3	Low
4	SNMP	192.168.10.71 and 192.168.10.33	As found, SNMP was not running on either machine (UDP 161 closed on both) — nothing to enumerate.	Figures 4, 5	Low
5	SNMP	192.168.10.33	After SNMP was enabled in the lab with the default community string 'public', a full snmpwalk extracted detailed system information with no other credential: OS build (Windows, build 19045, 64-bit), system name DESKTOP-8T992PD, interfaces, uptime, and (in the full output) processes and installed software.	Figure 8	High

5. Hardening Notes

Finding #	Problem	Hardening Fix
1	Error-code differences confirm that the default admin shares C$/ADMIN$ exist, even though access is denied.	Keep anonymous-access restrictions enabled. Restrict SMB (TCP 445) to trusted management hosts with the firewall, and disable the administrative shares (AutoShareServer) where they are not required.
2	No leak — the server already refuses anonymous enumeration. The risk is a future settings change silently opening it.	Keep 'RestrictAnonymous' and 'RestrictAnonymousSAM' enabled so user lists, shares and policy cannot be enumerated without authentication; audit these settings after updates or role changes.
3	No leak — zone transfers are already restricted. A misconfigured DNS server would hand an attacker the full network map.	Keep zone transfers set to 'Only to authorised name servers' (never 'to any server'), and log/alert on failed AXFR attempts.
4	No issue as found. The risk appears only if SNMP is switched on carelessly (see Finding 5).	Leave SNMP disabled on machines that do not need it. 'Service not installed' is the strongest SNMP hardening.
5	SNMPv2c with the default community string 'public' discloses full system details to anyone on the network; v2c also sends the community in clear text.	Never use default community strings — use a long, random community, or SNMPv3 with authentication and encryption. Restrict UDP 161 to the management machine only (firewall + 'accept packets from these hosts'), and disable SNMP entirely when monitoring is not required. For this lab VM, the 'public' community should be removed again (or SNMP disabled) now that the exercise is complete.

6. Observations

- One target, two behaviours: the domain controller (.71) refused every anonymous question (SMB and DNS), while the same Windows family answered everything once SNMP was enabled with its default community on .33. The difference is configuration, not the operating system.
- Error messages are information too. On .71 nothing was 'given away', yet the difference between ACCESS_DENIED and BAD_NETWORK_NAME still confirmed which shares exist — enumeration can leak through refusals, not only through answers.
- 'Closed', 'open|filtered' and a successful walk are three different SNMP states, and all three were observed in this project: service off (closed), service on but silent to a UDP probe (open|filtered), and service answering a correctly-keyed query (the walk).
- The troubleshooting path is itself evidence of method: firewall already allowed the service, the service was running, and the walk only succeeded after the community name was actually saved in the Accepted community names list and the service restarted — each step was verified by re-testing from Kali.

7. Result

All three enumeration areas were completed against the lab targets:

- SMB (.71): enumerated with NSE scripts and enum4linux — anonymously blocked; only default-share existence inferable.
- DNS (.71): zone transfer attempted with dig AXFR — refused, as a hardened server should.
- SNMP (.71, .33): found disabled on both; enabled deliberately on the student's own .33 VM and fully enumerated with snmpwalk using the default community 'public', demonstrating the leak and its fix.

Deliverables complete: this report with evidence screenshots at every step, the findings table, and a hardening fix for every finding, plus the saved output files (enum-smb-nmap.txt, enum-smb-full.txt, dns-zone-transfer.txt, enum-snmp-check.txt, enum-snmp-check-33.txt, snmp-walk.txt).

— End of Report —